



## Data Structures and Algorithms (23CSE203)

### Lab Manual

|                  |                       |
|------------------|-----------------------|
| NAME             | CH. JAI RAM CHARAN    |
| ROLL NO          | AV.SC.U4CSE25106      |
| YEAR/SEM/SECTION | 2nd YEAR/ SEM-3/CSE-B |
| CREDITS          | 05                    |

### Submitted By

|             |                             |
|-------------|-----------------------------|
| NAME        | DR. RAJ KUMAR BATCHU        |
| DESIGNATION | ASST.PROFFESSOR (Sl. Grade) |
| DEPARTMENT  | SCHOOL OF COMPUTING         |

### Submitted to:

## Index

| SNO | WEEK                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Page-no |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| 1   | a) A rocket launch system displays a count down before launch. Instead of using a loop, implement the count down using recursion. Write a recursive python program to display countdown from n to 1 followed by launch.<br>b) Banks calculate the growth of investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate $P^n$ where P = Principal growth factor, n = number of years<br>c) A HR department stores employee details in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.<br>d) In a logistic department, the no of possible ways to arrange n diff parcels are calculated using the factorial formula( $n!$ ). Write a recursive formula to find the factorial of a given number<br>e) Print first n terms in a fibanocci number series | 4-9     |
| 2   | a) Write the implementation for the linear search algorithm<br>b) Write the implementation for the binary search algorithm<br>c) Write the implementation for the binary search algorithm along with the sorting the input elements if they are not sorted.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 10-14   |
| 3   | a) Write a python program to implement bubble sort where the input elements are taken from the user.<br>b) Write a python program to implement insertion sort where the input elements are taken from the user.<br>c) Write a python program to implement selection sort where the input elements are taken from the user.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 15-18   |
| 4   | a) Write a python program to implement merge sort where the input elements are taken from the user.<br>b) Write a python program to implement quick sort where the input elements are taken from the user.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 19-22   |
| 5   | a) Write a Python program to implement a Singly Linked List (SLL) with the following operations:<br>b) Write a Python program to implement a Double Linked List (DLL) with the following operations:<br>c) Write a Python program to implement a Circular Linked List (CLL) with the following operations:<br>1. Create LL<br>2. Insert at Beginning<br>3. Insert at End<br>4. Insert at Index<br>5. Delete by Value<br>6. Delete Prior Node<br>7. Delete Last Node<br>8. Count Number of Nodes                                                                                                                                                                                                                                                                                                                                                                                                                                              | 23-45   |

|   |                                                                                                                                                                                                                                 |       |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
|   | 9. Display/Traverse<br>10. Exit                                                                                                                                                                                                 |       |
| 6 | a) Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Array.<br>b) Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using Linked List. | 46-52 |

## Lab – 1

1a)

### Aim :

- A rocket launch system displays a count down before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display count down from n to 1 followed by launch.

### Program:

```
#Count Down #CSE25106
def countdown(n):
    if n >= 0 :
        if n != 0:
            print(n)
            return countdown(n-1)
        else:
            print("Launch!")

n= int(input("Enter the number:"))
countdown(n)
```

### Output:

```
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jaira/OneDrive/Documents/Desktop/SEM-2/DSA/DSA25106/CSE25106_DSA/lab1_1.py
Enter the number:10
10
9
8
7
6
5
4
3
2
1
Launch!
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> █
```

### Error Table:

| Error                      | Rectification              |
|----------------------------|----------------------------|
| Output: None after launch! | Swapping if and else case. |

### Description:

- The program displays a countdown from a given number n to 1.

- It uses a recursive function named `countdown()`.
- The function calls itself with the value  $n-1$ .
- The recursion continues until  $n$  becomes 0.
- When  $n$  reaches 0, the program displays "**Launch!**".

1b)

### Aim:

- Banks Calculate the growth of investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate  $P^n$  where  $P$  = Principal growth amount,  $n$  = no of years

### Program:

```
#Growth #CSE25106
n = int(input("Enter number of years:"))
p = int(input("Enter annual principal:"))
def growth(p,n):
    if n < 0:
        return "Invalid input for number of years"
    if n == 0:
        return 1
    else:
        return p*growth(p,n-1)

print(growth(p,n))
```

### Output:

```
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jaira/OneDrive/Documents/Desktop/SEM-2/DSA/DSA25106/CSE25106_DSA/lab1_2.py
Enter number of years:1
Enter annual principal:300
300
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jaira/OneDrive/Documents/Desktop/SEM-2/DSA/DSA25106/CSE25106_DSA/lab1_2.py
Enter number of years:-6
Enter annual principal:66
Invalid input for number of years
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> █
```

### Error Table:

| Error                                            | Rectification                            |
|--------------------------------------------------|------------------------------------------|
| RecursionError: maximum recursion depth exceeded | Added if case to give valid input for n. |

### Description:

- P represents the principal/growth amount.
- n represents the number of years.
- The user enters the values of P and n.
- The `growth()` function calls itself with `n-1`.
- When n becomes 0, the function returns 1.
- The result is calculated by repeatedly multiplying P.

### 1c)

### Aim:

- A HR department stores employee details in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

### Program:

```
#Searching of element #CSE25106
list = []
t=int(input("Enter number of inputs:"))
for _ in range(t):
    n = int(input("Enter the ID:"))
    list.append(n)
ID = int(input("Enter ID to seach:"))
start = 0
def search(list,ID,start):
    if list[start] == ID:
        print(f"ID found at : {start}")
    elif list[start] == len(list):
        print("ID not found")
    else:
        return search(list,ID,start+1)

search(list,ID,start)
```

## Output:

```
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jaira/OneDrive/Documents/Desktop/SEM-2/DSA/DSA25106/CSE25106_DSA/lab1_3.py
Enter number of inputs:3
Enter the ID:1
Enter the ID:2
Enter the ID:3
Enter ID to seach:2
ID found at : 1
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jaira/OneDrive/Documents/Desktop/SEM-2/DSA/DSA25106/CSE25106_DSA/lab1_3.py
Enter number of inputs:3
Enter the ID:1
Enter the ID:2
Enter the ID:3
Enter ID to seach:4
ID not found
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> █
```

## Error Table:

| Error                                                    | Rectification                |
|----------------------------------------------------------|------------------------------|
| ValueError: invalid literal for int() with base 10: '23' | Should take a int value only |

## Description:

- The program stores employee IDs in a list.
- The user enters the number of employee IDs.
- Each employee ID is added to the list.
- The user enters the ID that needs to be searched.
- The `search()` function checks the elements recursively.
- If the ID matches, its position is displayed.
- If the ID is not found, the program displays "**ID not found**".

## 1d)

## Aim:

- In a logistic department, the no of possible ways to arrange n diff parcels are calculated using the factorial formula( $n!$ ). Write a recursive formula to find the factorial of a given number

## Program:

```
#Factorial #CSE25106
def factorial(n):
    if n > 0:
        if n == 1:
            return 1
        else:
            return n*factorial(n-1)
    else:
```

```

        return "Please enter a positive number"
n = int(input("Enter the number:"))
print(factorial(n))

```

## Output:

```

PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jaira/OneDrive/Desktop/SEM-3/DSA/DSA25106/CSE25106_DSA/lab1_4.py
Enter the number:5
120
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jaira/OneDrive/Desktop/SEM-3/DSA/DSA25106/CSE25106_DSA/lab1_4.py
Enter the number:-1
Please enter a positive number
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA>

```

## Error Table:

| Error                                    | Rectification                                   |
|------------------------------------------|-------------------------------------------------|
| Didn't implement for the negative values | Added else to make user to give positive input. |

## Description:

- The program calculates the factorial of a given number.
- Factorial is represented as  $n!$ .
- The `factorial()` function calls itself with  $n-1$ .
- If  $n$  is 1, the function returns 1.
- Otherwise, the current value of  $n$  is multiplied by the factorial of  $n-1$ .
- Negative or invalid values are handled by displaying a suitable message.

## 1e)

## Aim:

- Print first  $n$  terms in a fibonacci number

## Program:

```

#Fibonacci series #CSE25106
def fib(n):
    if n == 1:
        return 0
    if n == 2:
        return 1
    else:
        return fib(n-1) + fib(n-2)
def print_fib(n):
    for i in range(1, n+1):
        print(fib(i), end=" ")

```

```
n = int(input("Enter the number:"))
print_fib(n)
```

## Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jaira/OneDrive/Desktop/SEM-3/DSA/DSA25106/CSE25106_DSA/lab1_5.py
Enter the number:5
0 1 1 2 3
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Users/jaira/OneDrive/Desktop/SEM-3/DSA/DSA25106/CSE25106_DSA/lab1_5.py
Enter the number:-3
Please enter a positive number
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> █
```

## Error Table:

| Error                                                      | Rectification                                               |
|------------------------------------------------------------|-------------------------------------------------------------|
| Numbers are not printed if there are in the same function. | Added a new function with for loop for printing the numbers |

## Description:

- The program prints the first n terms of the Fibonacci series.
- The `fib()` function calculates Fibonacci numbers recursively.
- The first term is 0.
- The second term is 1.
- Each subsequent term is calculated by adding the previous two terms.
- The `print_fib()` function is used to print the Fibonacci terms.

## Lab – 2

2a)

### Aim:

- Write a program for the linear search algorithm

### Program:

```
# Linear search #CSE25106
arr=[]
n = int(input("Enter no of elements:"))
while n < 1:
    print("no of elements should be greater than 0")
    n = int(input("Enter no of elements:"))
for i in range(n):
    elements = int(input("Enter the elements:"))
    arr.append(elements)
key = int(input("Enter element to search:"))
def linear_search(arr,key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
        else:
            i+= 1
    return -1
result = linear_search(arr,key)
if result != -1 :
    print("Element found at index ",result)
else:
    print("Element not found")
```

### Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA\lab2_1.py"
Enter the elements:5
Enter element to search:3
Element found at index 1
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA\lab2_1.py"
Enter no of elements:3
Enter the elements:1
Enter the elements:2
Enter the elements:3
Enter element to search:4
Element not found
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA>
```

### Error Table:

| Error | Rectification |
|-------|---------------|
|-------|---------------|

Forgot to return at the end which lead to the program returning none when the element is not found

Returned -1 at the very end which fixed the element not found case

### Description:

- The program implements the Linear Search algorithm.
- The user enters the number of elements.
- The elements are stored in an array/list.
- The user enters the element to be searched.
- The `linear_search()` function checks the elements one by one.
- If the element matches the key, its index is returned.
- If the element is not found, -1 is returned.

2b)

### Aim:

Write a program for binary search algorithm considering the elements to be sorted

### Program:

```
# Binary search of sorted array #CSE25106
arr=[]
n = int(input("Enter no of elements:"))
for i in range(n):
    elements = int(input("Enter the elements in sorted order:"))
    arr.append(elements)
key = int(input("Enter element to search:"))
def binary_search(arr,key):
    low = 0
    high = len(arr) - 1
    while low < high:
        mid = (low+high)//2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

result = binary_search(arr,key)
if result != -1 :
    print("Element found at index :",result)
else:
```

```
print("Element not found")
```

## Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106_DSA\lab2_2.py"
Enter no of elements:3
Enter the elements in sorted order:1
Enter the elements in sorted order:2
Enter the elements in sorted order:3
Enter element to search:2
Element found at index : 1
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106_DSA\lab2_2.py"
Enter no of elements:4
Enter the elements in sorted order:1
Enter the elements in sorted order:2
Enter the elements in sorted order:3
Enter the elements in sorted order:4
Enter element to search:5
Element not found
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> 
```

## Error Table:

| Error                                                                           | Rectification                               |
|---------------------------------------------------------------------------------|---------------------------------------------|
| Wrote the mid formula outside the while loop which gave the infinite loop error | Added the mid formula inside the while loop |

## Description:

- The program implements Binary Search on a sorted array.
- The user enters elements in sorted order.
- The algorithm uses low, high, and mid values.
- The middle element is calculated using  $(low + high) // 2$ .
- If the middle element equals the search key, its index is returned.
- If the key is greater than the middle element, the right half is searched.
- If the key is smaller, the left half is searched.
- If the element is not found, -1 is returned.

## 2c)

## Aim:

Write a program for implementation of binary search along with the sorting the elements

## Program:

```
#Binary search of unsorted array. #CSE25106
arr = []
n = int(input("Enter no of elements:"))
for i in range(n):
    element = int(input("Enter the element:"))
```

```

    arr.append(element)
# To check and sort the list.
if arr == sorted(arr):
    print("\n The input list is already sorted")
else:
    print("\nThe input list is not sorted")
    print("\nSorting the list....")
    arr.sort()
print("\nThe sorted list is:",arr)
key = int(input("Enter element to search:"))
def binary_search(arr,key):
    low = 0
    high = len(arr) - 1
    while low < high:
        mid = (low+high)//2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

result = binary_search(arr,key)
if result != -1 :
    print("Element found at index:",result)
else:
    print("Element not found")

```

## Output:

```

PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_D
SA\lab2_3.py"
Enter no of elements:3
Enter the element:1
Enter the element:3
Enter the element:2

The input list is not sorted

Sorting the list....

The sorted list is: [1, 2, 3]
Enter element to search:2
Element found at index: 1
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_D
SA\lab2_3.py"
Enter no of elements:3
Enter the element:1
Enter the element:3
Enter the element:2

The input list is not sorted

Sorting the list....

The sorted list is: [1, 2, 3]
Enter element to search:4
Element not found
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA>

```

### Error Table:

| Error                                                                                                    | Rectification                                                                          |
|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Used sorted method instead of which need to be referred to another variable as it returns a sorted array | Used the arr.sort() method which directly sorts the arr without using another variable |

### Description:

- The program performs Binary Search on user-provided elements.
- The input elements may initially be unsorted.
- The program checks whether the list is already sorted.
- If the list is not sorted, `arr.sort()` is used.
- The sorted list is displayed.
- The user enters the element to be searched.
- Binary Search is then performed on the sorted list.
- The middle element is compared with the search key.
- If the element is found, its index is displayed.
- Otherwise, "**Element not found**" is displayed.

## Lab – 3

### 3a)

#### Aim:

- Write a python program to implement bubble sort where the input elements are taken from the user.

#### Program:

```
# BUBBLE SORT # CSE25106
def main():
    arr = []
    n = int(input("Enter number of elements: "))
    for i in range(n):
        num = int(input(f"Enter the {i+1} element: "))
        arr.append(num)
    bubble_sort(arr)
    print("Sorted array:", arr)
```

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

if __name__ == '__main__':
    main()
```

## Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106_DSA\lab3_1.py"
Enter number of elements: 4
Enter the 1 element: 1
Enter the 2 element: 4
Enter the 3 element: 6
Enter the 4 element: 2
Sorted array: [1, 2, 4, 6]
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA>
```

## Error Table:

| Error                                | Rectification                                                  |
|--------------------------------------|----------------------------------------------------------------|
| Gave the numbers in descending order | Changed the sign symbol while comparing the adjacent elements. |

## Description:

- The program implements the Bubble Sort algorithm.
- The user enters the number of elements.
- The elements are stored in a list.
- Adjacent elements are compared with each other.
- If the first element is greater than the second element, they are swapped.
- This process is repeated for multiple passes.
- Larger elements gradually move toward the end of the list.
- The `bubble_sort()` function performs the sorting.

## 3b)

## Aim:

- Write a python program to implement insertion sort where the input elements are taken from the user.

## Program:

```
# INSERTION SORT # CSE25106
```

```

def main():
    arr = []
    n = int(input("Enter number of elements: "))
    for i in range(n):
        num = int(input(f"Enter the {i+1} element: "))
        arr.append(num)
    insertion_sort(arr)
    print(arr)

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j = j-1
        arr[j+1] = key

if __name__ == '__main__':
    main()

```

## Output:

```

PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA\lab3_2.py"
Enter number of elements: 3
Enter the 1 element: 1
Enter the 2 element: 4
Enter the 3 element: 2
[1, 2, 4]

```

## Error Table:

| Error                                              | Rectification       |
|----------------------------------------------------|---------------------|
| If range is taken incorrect, invalid output comes. | Take correct range. |

## Description:

- The program implements the Insertion Sort algorithm.
- The user enters the number of elements.
- The elements are stored in a list.
- The first element is considered as the initially sorted portion.
- Each subsequent element is stored in a variable called **key**.
- The **key** is compared with the elements before it.
- Elements greater than the **key** are shifted one position to the right.
- The **key** is inserted into its correct position.
- This process continues until all elements are sorted.

### 3c)

#### Aim:

- Write a python program to implement selection sort where the input elements are taken from the user.

#### Program:

```
# SELECTION SORT # CSE25106
arr = []
n = int(input("Enter number of elements: "))
for i in range(n):
    num = int(input(f"Enter the {i+1} element: "))
    arr.append(num)
selection_sort(arr)
rint(arr)

def selection_sort(arr):
    for i in range(len(arr)-1):
        min_index = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
```

#### Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_D
SA\lab3_1.py"
Enter number of elements: 4
Enter the 1 element: 1
Enter the 2 element: 3
Enter the 3 element: 6
Enter the 4 element: 2
[1, 2, 3, 6]
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_D
SA\lab3_1.py"
Enter number of elements: 3
Enter the 1 element: 1
Enter the 2 element: 5
Enter the 3 element: 5
[1, 5, 5]
❖ PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA>
```

#### Error Table:

| Error                | Rectification                                                                         |
|----------------------|---------------------------------------------------------------------------------------|
| Wrong comparison.(>) | Use <code>arr[j] &lt; arr[min_index]</code> and initialize <code>min_index = i</code> |

**Description:**

- The program implements the Selection Sort algorithm.
- The user enters the number of elements.
- The elements are stored in a list.
- The algorithm starts from the first unsorted element.
- The smallest element in the remaining list is identified.
- Its index is stored in `min_index`.
- The smallest element is swapped with the current element.
- The process is repeated for all positions in the list.

**Lab – 4**

4a)

**Aim:**

- Write a python program to implement merge sort where the input elements are taken from the user.

**Program:**

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]

    left = merge_sort(left)
    right = merge_sort(right)

    i = 0
    j = 0
    k = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
```

```

        arr[k] = left[i]
        i += 1
    else:
        arr[k] = right[j]
        j += 1
    k += 1

while i < len(left):
    arr[k] = left[i]
    i += 1
    k += 1

while j < len(right):
    arr[k] = right[j]
    j += 1
    k += 1

return arr

n = int(input("Enter size of array: "))

arr = []

for i in range(n):
    arr.append(int(input(f"Enter {i+1} element: ")))

merge_sort(arr)

print("Sorted array:")

for element in arr:
    print(element, end=" ")

```

### Output:

```

PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-4\lab4_1.py"
Enter size of array: 4
Enter 1 element: 1
Enter 2 element: 5
Enter 3 element: 3
Enter 4 element: 8
Sorted array:
1 3 5 8
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> -1
-1
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>

```

### Error Table:

| Error | Rectification |
|-------|---------------|
|-------|---------------|

|                                                   |                                        |
|---------------------------------------------------|----------------------------------------|
| TypeError: object of type 'NoneType' has no len() | Taken return arr at end of merge_sort. |
|---------------------------------------------------|----------------------------------------|

### Description:

- The program implements the **Merge Sort** algorithm.
- The user enters the size of the array.
- The elements are taken from the user and stored in a list.
- The `merge_sort()` function recursively divides the array into two halves.
- The array is divided until each sub-array contains one or zero elements.
- The left and right portions are then sorted recursively.
- The sorted left and right arrays are compared element by element.
- The smaller element is placed into the original array.
- Remaining elements from either portion are then copied into the array.
- Finally, the completely sorted array is displayed.
- Merge Sort follows the **divide-and-conquer** approach.

4b)

### Aim:

- Write a python program to implement quick sort where the input elements are taken from the user.

### Program:

```
def quick_sort(a, low, high):
    if low < high:
        i = low
        j = high
        pivot = low

        while i < j:
            while i < len(a) and a[i] < a[pivot]:
                i += 1
            while a[j] > a[pivot]:
                j -= 1
            if i < j:
                a[i], a[j] = a[j], a[i]
            a[j], a[pivot] = a[pivot], a[j]
            quick_sort(a, low, j-1)
            quick_sort(a, j+1, high)

n = int(input("Enter size of array: "))
arr = []
for i in range(n):
    arr.append(int(input(f"enter {i+1} element: ")))
quick_sort(arr, 0, n-1)
```

```
print(arr)
```

### Output:

```

PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-4\lab4_2.py"
Enter size of array: 4
enter 1 element: 1
enter 2 element: 7
enter 3 element: 3
ent
[1, Focus folder in explorer (ctrl + click)
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> -1
-1
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>

```

### Error Table:

| Error                                                                            | Rectification                                                           |
|----------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <code>i &lt; len(a)</code> checks the complete array instead of current subarray | Use <code>i &lt;= high</code> and maintain correct partition boundaries |

### Description:

- The program implements the **Quick Sort** algorithm.
- The user enters the size of the array.
- The elements are taken from the user and stored in a list.
- The `quick_sort()` function is used to sort the array recursively.
- The first element of the current portion is selected as the **pivot**.
- Two pointers, `i` and `j`, are used to compare elements with the pivot.
- `i` moves from the left side and searches for an element greater than or equal to the pivot.
- `j` moves from the right side and searches for an element smaller than or equal to the pivot.
- If `i < j`, the two elements are swapped.
- After the partitioning process, the pivot is placed in its correct position.
- The array is then divided into left and right portions.
- Quick Sort is recursively applied to both portions.
- Finally, the sorted array is printed.

## Week-5

5a)

Aim:

- Write a Python program to implement a Singly Linked List (SLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

```
print("Implementation of SLL")  
  
class Node:
```

```

def __init__(self,data):
    self.data = data
    self.next = None

class SingleLinkedList:
    def __init__(self):
        self.head = None
    #Creation
    def create(self):
        n = int(input("Enter no of elements:"))
        for i in range(n):
            data = int(input("Enter the values:"))
            new = Node(data)
            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new
    #Insert begin
    def insert_begin(self,data):
        new = Node(data)
        new.next = self.head
        self.head = new
    #Insert end
    def insert_end(self,data):
        new = Node(data)
        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new
    #Insert at specific index
    def insert_index(self,index,data):
        if index == 0:
            self.insert_begin(data)
            return
        elif(index < 0):
            print("Invalid index")
            return
        else:
            new = Node(data)
            temp = self.head
            for i in range(index-1):
                if temp.next is None:

```

```

        print("Invalid index")
        return
    temp = temp.next
    new.next = temp.next
    temp.next = new
#Delete specific value
def delete(self,data):
    if self.head is None:
        print("No data")
    else:
        temp = self.head
        if temp and temp.data == data:
            self.head = temp.next
            print("Value deleted")
            return
        while temp.next and temp.next.data != data:
            temp = temp.next
        if temp.next is None:
            print("Value not present")
        else:
            temp.next = temp.next.next
            print("Value deleted")
#Delete begin
def delete_begin(self):
    if self.head is None:
        print("No data to delete")
    else:
        temp = self.head
        self.head = temp.next
        print("Deleted value = ",temp.data)
#Delete end
def delete_end(self):
    if self.head is None:
        print("No data to delete")
    elif self.head.next is None:
        self.head = None
    else:
        temp = self.head
        temp1 = temp
        while temp.next:
            temp1 = temp
            temp = temp.next
        temp1.next = None
#Count
def count(self):
    if self.head is None:
        print("No data to count")
    else:
        count = 0

```

```

        temp = self.head
        while temp:
            count += 1
            temp = temp.next
        print(f"Number of nodes = {count}")
#Display
def display(self):
    if self.head is None:
        print("No data")
    else:
        temp = self.head
        while temp:
            print(temp.data,end="-->")
            temp = temp.next
        print("None")
#Calling main function
SLL = SingleLinkedList()

while True:
    print("1.Create SLL")
    print("2.Insert at beginning")
    print("3.Insert at ending")
    print("4.Insert at index")
    print("5.Delete by value")
    print("6.Delete first node")
    print("7.Delete last node")
    print("8.Count no of nodes")
    print("9.Display / Traverse")
    print("10.Exit")

    choice = int(input("Enter your choice:"))

    if(choice == 1):
        SLL.create()
        print("SLL is created")
    elif(choice == 2):
        data = int(input("Enter data to insert: "))
        SLL.insert_begin(data)
        print("Data is inserted at beginning")
    elif(choice == 3):
        data = int(input("Enter data to insert: "))
        SLL.insert_end(data)
        print("Data is inserted at ending")
    elif(choice == 4):
        data = int(input("Enter data to insert: "))
        index = int(input("Enter the index:"))
        SLL.insert_index(index,data)
        print("Data is inserted at specific index")

```

```
elif(choice == 5):
    data = int(input("Enter data to insert: "))
    SLL.delete(data)
elif(choice == 6):
    SLL.delete_begin()
elif(choice == 7):
    SLL.delete_end()
elif(choice == 8):
    SLL.count()
elif(choice == 9):
    SLL.display()
elif(choice == 10):
    print("Exiting the program")
    break
else:
    print("Invalid choice")
```

Output:

```

PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-5\lab5_1.py"
Implementation of SLL
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:1
Enter no of elements:2
Enter the values:1
Enter the values:2
SLL is created
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:2
Enter data to insert: 0
Data is inserted at beginning
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:3
Enter data to insert: 3
Data is inserted at ending
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:9
0-->1-->2-->3-->None
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:8
Number of nodes = 4
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:10
Exiting the program
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>

```

**Error Table:**

| Error                                                                                                         | Rectification                                                                        |
|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| The program may display an incorrect message for insertion at a specific index even when the index is invalid | Check whether the index is valid before displaying the successful insertion message. |

**Description:**

- The program implements a **Singly Linked List** using Python classes.
- The **Node** class is used to create individual nodes.
- Each node contains two parts: **data** and **next**.
- **data** stores the value of the node.
- **next** stores the reference to the next node.
- The **SingleLinkedList** class manages the linked list.
- The **head** variable stores the reference to the first node.
- The program provides an option to **create** a linked list.
- It supports **insertion at the beginning** of the list.
- It supports **insertion at the end** of the list.
- It supports **insertion at a specific index**.
- It supports **deletion of a specific value**.
- It supports **deletion of the first node**.
- It supports **deletion of the last node**.
- It provides a function to **count the number of nodes**.
- It provides a function to **display/traverse** all the nodes.
- A menu-driven approach is used to allow the user to select different operations.
- The program continues running until the user selects the **Exit** option.

5b)

**Aim:**

- Write a Python program to implement a Double Linked List (DLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value

6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

```
print("Implementation of DLL")

class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoubleLinkedList:
    def __init__(self):
        self.head = None
    # Creation
    def create(self):
        n = int(input("Enter no of elements: "))
        for i in range(n):
            data = int(input("Enter the values: "))
            new = Node(data)
            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new
                new.prev = temp

    # Insert begin
    def insert_begin(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
        else:
            new.next = self.head
            self.head.prev = new
```

```

        self.head = new

# Insert end
def insert_end(self, data):
    new = Node(data)
    if self.head is None:
        self.head = new
    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new
        new.prev = temp

# Insert at specific index
def insert_index(self, index, data):
    if index < 0:
        print("Invalid index")
        return
    if index == 0:
        self.insert_begin(data)
        return
    if self.head is None:
        print("Invalid index")
        return
    temp = self.head
    for i in range(index - 1):
        if temp.next is None:
            print("Invalid index")
            return
        temp = temp.next
    new = Node(data)
    new.next = temp.next
    new.prev = temp
    if temp.next is not None:
        temp.next.prev = new
    temp.next = new

# Delete specific value
def delete(self, data):
    if self.head is None:
        print("No data")
        return
    temp = self.head
    while temp and temp.data != data:
        temp = temp.next
    if temp is None:
        print("Value not present")

```

```

    else:
        if temp.prev is not None:
            temp.prev.next = temp.next
        else:
            self.head = temp.next
        if temp.next is not None:
            temp.next.prev = temp.prev
        print("Value deleted")

# Delete begin
def delete_begin(self):
    if self.head is None:
        print("No data to delete")
    else:
        temp = self.head
        self.head = temp.next
        if self.head is not None:
            self.head.prev = None
        print("Deleted value = ", temp.data)

# Delete end
def delete_end(self):
    if self.head is None:
        print("No data to delete")
    elif self.head.next is None:
        print("Deleted value = ", self.head.data)
        self.head = None

    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        print("Deleted value = ", temp.data)
        temp.prev.next = None

# Count
def count(self):
    if self.head is None:
        print("No data to count")
    else:
        count = 0
        temp = self.head
        while temp:
            count += 1
            temp = temp.next
        print(f"Number of nodes = {count}")

# Display / Traverse

```

```

def display(self):
    if self.head is None:
        print("No data")
    else:
        temp = self.head
        while temp:
            print(temp.data, end=" <--> ")
            temp = temp.next

        print("None")

DLL = DoubleLinkedList()

while True:
    print("\n1.Create DLL")
    print("2.Insert at beginning")
    print("3.Insert at ending")
    print("4.Insert at index")
    print("5.Delete by value")
    print("6.Delete first node")
    print("7.Delete last node")
    print("8.Count no of nodes")
    print("9.Display / Traverse")
    print("10.Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        DLL.create()
        print("DLL is created")

    elif choice == 2:
        data = int(input("Enter data to insert: "))
        DLL.insert_begin(data)
        print("Data is inserted at beginning")

    elif choice == 3:
        data = int(input("Enter data to insert: "))
        DLL.insert_end(data)
        print("Data is inserted at ending")

    elif choice == 4:
        data = int(input("Enter data to insert: "))
        index = int(input("Enter the index: "))
        DLL.insert_index(index, data)

```

```
elif choice == 5:
    data = int(input("Enter data to delete: "))
    DLL.delete(data)

elif choice == 6:
    DLL.delete_begin()

elif choice == 7:
    DLL.delete_end()

elif choice == 8:
    DLL.count()

elif choice == 9:
    DLL.display()

elif choice == 10:
    print("Exiting the program")
    break

else:
    print("Invalid choice")
```

Output:

```

PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-5\lab5_2.py"
Implementation of DLL

1.Create DLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 1
Enter no of elements: 2
Enter the values: 1
Enter the values: 2
DLL is created

1.Create DLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 2
Enter data to insert: 0
Data is inserted at beginning

```

Error Table:

| Error                                                                              | Rectification                                             |
|------------------------------------------------------------------------------------|-----------------------------------------------------------|
| The prev and next links may not be updated correctly during insertion or deletion. | Update both links whenever a node is inserted or deleted. |

## Description:

- The program implements a **Doubly Linked List** using Python classes.
- The **Node** class is used to create individual nodes.
- Each node contains three parts: **data**, **prev**, and **next**.
- **data** stores the value of the node.
- **prev** stores the reference to the previous node.
- **next** stores the reference to the next node.
- The **DoubleLinkedList** class is used to manage the linked list.
- The **head** variable stores the reference to the first node.
- The program provides an option to **create** a doubly linked list.
- It supports **insertion at the beginning** of the list.
- It supports **insertion at the end** of the list.
- It supports **insertion at a specific index**.
- It supports **deletion of a specific value**.
- It supports **deletion of the first node**.
- It supports **deletion of the last node**.
- It provides a function to **count the number of nodes**.

- It provides a function to **display/traverse** the list.
- During insertion and deletion, both **prev** and **next** links are updated.
- The list is displayed using <--> to show the connection between nodes.
- A **menu-driven approach** is used to perform different DLL operations.
- The program continues until the user selects the **Exit** option.

5c)

Aim:

- Write a Python program to implement a Circular Linked List (CLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

```
print("Implementation of CLL")

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
```

```

def __init__(self):
    self.head = None

# Creation
def create(self):
    n = int(input("Enter no of elements: "))

    for i in range(n):
        data = int(input("Enter the values: "))
        new = Node(data)

        if self.head is None:
            self.head = new
            new.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new
            new.next = self.head

# Insert begin
def insert_begin(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
        new.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        new.next = self.head
        self.head = new
        temp.next = self.head

# Insert end
def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
        new.next = self.head

```

```

else:
    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    temp.next = new
    new.next = self.head

# Insert at specific index
def insert_index(self, index, data):
    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_begin(data)
        return

    if self.head is None:
        print("Invalid index")
        return

    temp = self.head

    for i in range(index - 1):
        if temp.next == self.head:
            print("Invalid index")
            return

        temp = temp.next

    new = Node(data)
    new.next = temp.next
    temp.next = new

# Delete specific value
def delete(self, data):
    if self.head is None:
        print("No data")
        return

    # If head contains the value
    if self.head.data == data:

        # Only one node
        if self.head.next == self.head:

```

```

        self.head = None
        print("Value deleted")
        return

    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    temp.next = self.head.next
    self.head = self.head.next

    print("Value deleted")
    return

    temp = self.head

    while temp.next != self.head and temp.next.data != data:
        temp = temp.next

    if temp.next == self.head:
        print("Value not present")
    else:
        temp.next = temp.next.next
        print("Value deleted")

# Delete begin
def delete_begin(self):
    if self.head is None:
        print("No data to delete")
        return

    # Only one node
    if self.head.next == self.head:
        print("Deleted value = ", self.head.data)
        self.head = None
        return

    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    print("Deleted value = ", self.head.data)

    temp.next = self.head.next
    self.head = self.head.next

```

```

# Delete end
def delete_end(self):
    if self.head is None:
        print("No data to delete")
        return

    # Only one node
    if self.head.next == self.head:
        print("Deleted value = ", self.head.data)
        self.head = None
        return

    temp = self.head

    while temp.next.next != self.head:
        temp = temp.next

    print("Deleted value = ", temp.next.data)

    temp.next = self.head

# Count
def count(self):
    if self.head is None:
        print("No data to count")
        return

    count = 0
    temp = self.head

    while True:
        count += 1
        temp = temp.next

        if temp == self.head:
            break

    print(f"Number of nodes = {count}")

# Display / Traverse
def display(self):
    if self.head is None:
        print("No data")
        return

    temp = self.head

```

```

        while True:
            print(temp.data, end="-->")
            temp = temp.next

            if temp == self.head:
                break

        print("HEAD")

# Calling main function
CLL = CircularLinkedList()

while True:
    print("\n1.Create CLL")
    print("2.Insert at beginning")
    print("3.Insert at ending")
    print("4.Insert at index")
    print("5.Delete by value")
    print("6.Delete first node")
    print("7.Delete last node")
    print("8.Count no of nodes")
    print("9.Display / Traverse")
    print("10.Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        CLL.create()
        print("CLL is created")

    elif choice == 2:
        data = int(input("Enter data to insert: "))
        CLL.insert_begin(data)
        print("Data is inserted at beginning")

    elif choice == 3:
        data = int(input("Enter data to insert: "))
        CLL.insert_end(data)
        print("Data is inserted at ending")

    elif choice == 4:
        data = int(input("Enter data to insert: "))
        index = int(input("Enter the index: "))
        CLL.insert_index(index, data)

```

```
elif choice == 5:
    data = int(input("Enter data to delete: "))
    CLL.delete(data)

elif choice == 6:
    CLL.delete_begin()

elif choice == 7:
    CLL.delete_end()

elif choice == 8:
    CLL.count()

elif choice == 9:
    CLL.display()

elif choice == 10:
    print("Exiting the program")
    break

else:
    print("Invalid choice")
```

Output:

```

PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-5\lab5_3.py"
Implementation of CLL

1.Create CLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 1
Enter no of elements: 2
Enter the values: 1
Enter the values: 2
CLL is created

1.Create CLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 9
1-->2-->HEAD

1.Create CLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 10
Exiting the program
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>

```

## Error Table:

| Error                                           | Rectification                       |
|-------------------------------------------------|-------------------------------------|
| No major error is present in the given program. | No major rectification is required. |

## Description:

- The program implements a **Circular Linked List** using Python classes.
- The **Node** class is used to create individual nodes.
- Each node contains two parts: **data** and **next**.
- **data** stores the value of the node.
- **next** stores the reference to the next node.
- In a Circular Linked List, the last node points back to the first node.
- The **CircularLinkedList** class is used to manage the linked list.
- The **head** variable stores the reference to the first node.
- The program provides an option to **create** a circular linked list.
- It supports **insertion at the beginning** of the list.

- It supports **insertion at the end** of the list.
- It supports **insertion at a specific index**.
- It supports **deletion of a specific value**.
- It supports **deletion of the first node**.
- It supports **deletion of the last node**.
- It provides a function to **count the number of nodes**.
- It provides a function to **display/traverse** the circular linked list.
- During traversal, the program stops when it reaches the **head** again.
- The list is displayed using - - > and ends with **HEAD**, showing its circular nature.
- The program handles the special case where the list contains only one node.
- A **menu-driven approach** is used to perform different CLL operations.
- The program continues until the user selects the **Exit** option.

## Week-6

6a)

Aim:

- Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Array.

Program:

```
class Stack:
    def __init__(self,size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self,item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item,"pushed into the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item,"popped from the stack")

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:",self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack:")
            for i in range(self.top,-1,-1):
                print(self.stack[i])

size = int(input("Enter the size of the stack:"))
s = Stack(size)
while True:
    print("--STACK MENU--")
    print("1.Push")
    print("2.Pop")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")
```

```
choice = int(input("Enter your choice:"))

if choice == 1:
    item = int(input("Enter the element to push:"))
    s.push(item)
elif choice == 2:
    s.pop()
elif choice == 3:
    s.peak()
elif choice == 4:
    s.display()
elif choice == 5:
    print("Exiting program")
    break
else:
    print("Invalid choice")
```

Output:

```

PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-6\lab6_1.py"
Enter the size of the stack:5
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:1
Enter the element to push:1
1 pushed into the stack
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:1
Enter the element to push:2
2 pushed into the stack
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:2
2 popped from the stack
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:3
Top element: 1
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:4
The elements of the stack:
1
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:5
Exiting program
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>

```

## Error Table:

| Error                                                                                 | Rectification                                     |
|---------------------------------------------------------------------------------------|---------------------------------------------------|
| In peek, self.peek is taken in 'if' does not give correct answer for stack underflow. | Self.top is taken in 'if' to give correct output. |

## Description:

- The Stack class implements a stack using a Python list.
- `size` stores the maximum size of the stack.
- `stack` stores the stack elements.
- `top` keeps track of the top position. Initially, `top = -1`, meaning the stack is empty.
- **Push** adds an element to the top of the stack.
- **Pop** removes the top element from the stack.

- **Peek** displays the top element without removing it.
- **Display** prints all elements from top to bottom.
- **Stack Overflow** occurs when trying to push into a full stack.
- **Stack Underflow** occurs when trying to pop from an empty stack.
- The program uses a menu-driven approach with options for Push, Pop, Peek, Display, and Exit.

6b)

Aim:

- Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Linked List.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    def push(self, item):
        new = Node(item)
        new.next = self.top
        self.top = new
        print(item, "pushed into the stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp = self.top
            self.top = self.top.next
            print(temp.data, "popped from the stack")

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
```

```

        print("Stack is empty")
    else:
        print("The elements of the stack:")
        temp = self.top
        while temp is not None:
            print(temp.data)
            temp = temp.next

s = Stack()

while True:
    print("\n--STACK MENU--")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Exiting program")
        break
    else:
        print("Invalid choice")

```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-6\lab6_2.py"
```

```
--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 1
1 pushed into the stack
```

```
--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 2
2 pushed into the stack
```

```
--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the stack:
2
1
```

```
--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
2 popped from the stack
```

```
--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the stack:
1
```

```
--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 1
```

```
--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting program
```

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> 
```

## Error Table:

| Error                                             | Rectification |
|---------------------------------------------------|---------------|
| No major error in the linked-list implementation. | Not required. |

**Description:**

- The stack is implemented using a **singly linked list**.
- Each node contains **data** and a **next** pointer.
- **top** points to the top node of the stack.
- **Push** inserts a new node at the beginning of the linked list.
- **Pop** removes the node pointed to by **top**.
- **Peek** displays the data of the top node without removing it.
- **Display** traverses the linked list from **top** to the last node.
- If **top** is **None**, the stack is empty.
- Stack Overflow is not a fixed-size issue here because the stack grows dynamically, subject to available memory.